

Software-Architektur als Schichtendarstellung

Analyse von Software-Architektur mit S202

Johannes Weigend

Finale Fassung, 26. Mai 2026

Zusammenfassung

S202 berechnet aus Bytecode eine hierarchische Schichtendarstellung von Java-Systemen. Das Werkzeug leitet eine Architekturhypothese aus den beobachteten Klassenabhängigkeiten ab, macht Zyklen und Rückkanten als explizite Befunde sichtbar und prüft die fertige Darstellung redundant gegen Konsistenzregeln. Dieses Dokument beschreibt die typischen Fehler bei der Berechnung solcher Layouts, erklärt die konzeptionelle Lösung und zeigt, wie die Darstellung als Arbeitswerkzeug für Refactoring-Planung genutzt werden kann.

1 Einleitung

Software-Architektur wird seit langem als geschichtete Abhängigkeitsstruktur dargestellt. Die Grundidee ist einfach: Ein Element, das ein anderes Element benutzt, steht oberhalb des benutzten Elements. Abhängigkeiten laufen damit von oben nach unten. Kanten in Gegenrichtung sind auffällig, weil sie häufig auf Zyklen, unerwünschte Kopplung oder eine gebrochene Schichtenordnung hinweisen.

Solche vertikal ausgerichteten Graphen helfen, Abhängigkeiten in einem System sichtbar zu machen. Ein einfaches Beispiel liefern die Werkzeuge `jdeps` und `dot`: `jdeps` extrahiert die Klassenabhängigkeiten aus einem Jar, `dot` erzeugt daraus einen gerichteten Graphen. Abbildung 1 zeigt zunächst nur den azyklischen Teil des Example-Jars.

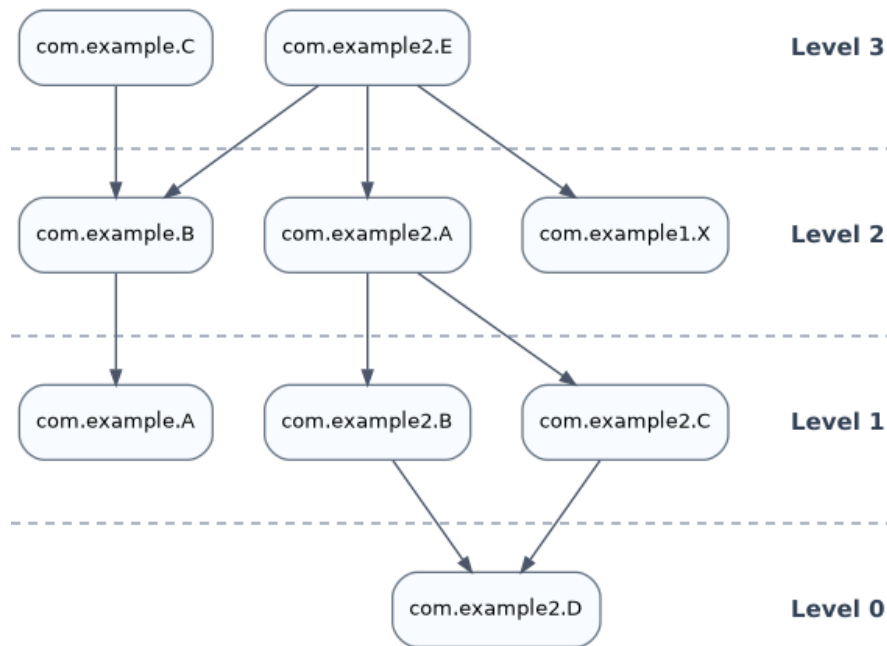


Abbildung 1: Azyklischer Ausschnitt aus `test-example-1.0.0.jar`. Die Pfeile zeigen von der nutzenden Klasse zur genutzten Klasse; die horizontalen Linien markieren die berechneten Levels.

Die Pfeile zwischen den Boxen sind Abhängigkeiten von Klassen durch Aufruf oder Nutzung. Im Beispiel nutzt `com.example.B` die Klasse `com.example.A`, kurz $B \rightarrow A$. Die horizontalen Linien sind manuell ergänzt und visualisieren die Ebenen der Schichtung. Level 0 bildet das Fundament; höhere Levels stehen für Elemente, die von darunterliegenden Elementen abhängen.

Diese Art der Darstellung ist nützlich und weit verbreitet. Als Visualisierung realer Anwendungen hat sie aber drei praktische Probleme:

1. Viele Pfeile machen die Grafik schon bei mittelgroßen Anwendungen unübersichtlich.
2. Die Paketstruktur ist über mehrere Levels verteilt und bildet keine sichtbare Gruppierung.
3. Bei zyklischen Abhängigkeiten muss der Zyklus für die Levelberechnung aufgebrochen werden, damit überhaupt eine vertikale Ordnung entsteht. Der ursprüngliche Abhängigkeitsgraph bleibt unverändert; verändert wird der daraus abgeleitete DAG (*Directed Acyclic Graph*, ein gerichteter Graph ohne Zyklen), auf dem die Levels berechnet werden. Entscheidend ist deshalb, welche Kante in diesem Berechnungsgraphen gesondert behandelt wird.

2 S202 als hierarchische Schichtendarstellung

Mit dem Open-Source-Werkzeug S202 versuchen wir, diese Probleme gezielt zu adressieren. S202 ersetzt den flachen Klassengraphen nicht einfach durch eine hübschere Darstellung, sondern durch eine hierarchische Architekturhypothese: Pakete bleiben als Container sichtbar, und innerhalb jedes Containers werden Pakete oder Klassen nach ihren Abhängigkeiten angeordnet.

Die Pfeile können optional eingeblendet werden. Im Normalfall soll die vertikale Ordnung aber bereits ohne Pfeile verständlich machen, welche Teile der Anwendung weiter oben liegen und welche als Fundament dienen. Unerwünschte Abhängigkeiten fallen dadurch nicht als einzelne Linie in einem großen Netz auf, sondern als Bruch in einer erwarteten Schichtenordnung.

Im Gegensatz zum flachen Klassengraphen sind die Levels bei S202 hierarchisch durch die Paketstruktur gegliedert. Jedes Paket ordnet seine direkten Kinder, also Subpakete oder Klassen, nach deren lokalen Abhängigkeiten. Die Paketstruktur bleibt dadurch sichtbar, statt im Klassengraphen auseinandergezogen zu werden.

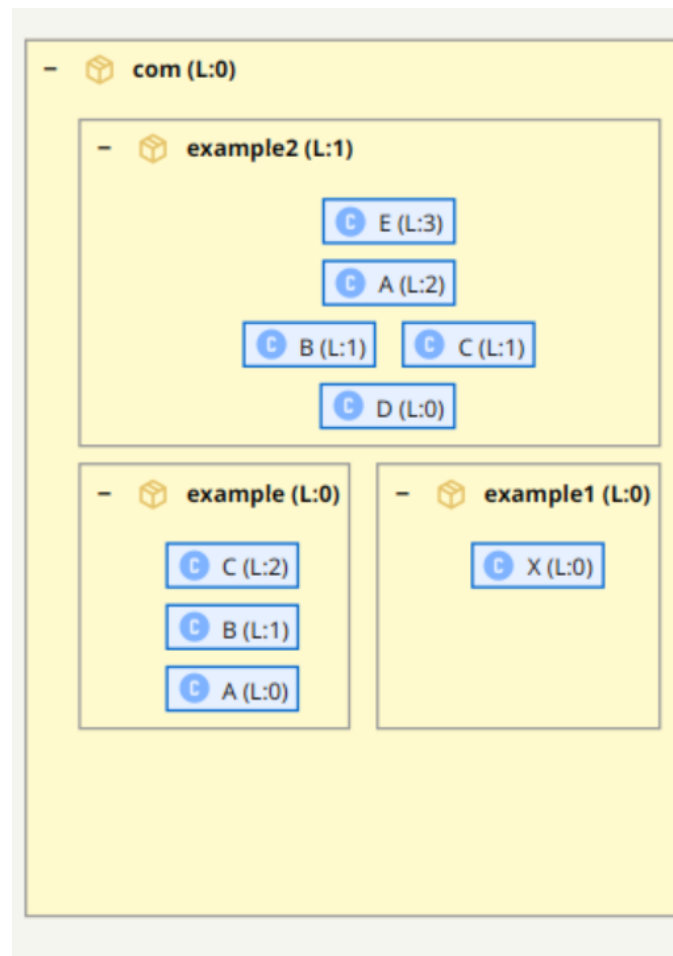


Abbildung 2: S202-Darstellung des azyklischen Beispielausschnitts: Pakete bleiben als Container sichtbar, Klassen werden innerhalb ihres Containers geordnet, und Pfeile können bei Bedarf eingeblendet werden.

Das Beispiel zeigt die innere Klassenhierarchie im Paket `example2` und zugleich die übergreifende Struktur: `example2` hängt von `example` und `example1` ab. Auf Klassenebene gilt für `example2`: $E \rightarrow A \rightarrow (B \mid C) \rightarrow D$. Die Darstellung abstrahiert dabei bewusst von einzelnen Kanten. Ob A eine Abhängigkeit nach B, nach C oder nach beiden hat, ist für die grobe Architekturansicht weniger wichtig als die Schichtung des Pakets. Genau diese Abstraktion macht umfangreiche Beziehungsnetze noch lesbar.

Die Idee ist nicht neu. Das kommerzielle Produkt Structure101 hatte bereits eine sehr ähnliche Darstellung. Es gibt jedoch keine offene Implementierung und keine technische Dokumentation, die erklärt, wie man ein solches Layout zuverlässig berechnet. S202 schließt diese Lücke als Open-Source-Werkzeug unter der Apache-Lizenz.

Dieses Dokument beschreibt die typischen Fallstricke bei der Realisierung eines solchen Layouts und zeigt eine Lösung, mit der sich viele dieser Probleme konzeptionell sauber eliminieren lassen.

3 Typische Fehler bei der Berechnung von Schichten

Die Abbildungen 1 und 2 zeigen bewusst einen einfachen, azyklischen Ausschnitt. Für diesen Fall ist die Schichtung eindeutig: E steht über A, A steht über B und C, und beide stehen über D. Es gibt keine Rückkopplung, deshalb muss keine Kante ausgewählt werden, die für die Levelberechnung ignoriert wird.

In realen Anwendungen ist dieser Fall eher die Ausnahme. Sobald Klassen oder Pakete zyklisch voneinander abhängen, reicht ein normaler Durchlauf über den Graphen nicht mehr aus. Genau dort beginnen die interessanten Fehler: Die Darstellung sieht oft noch plausibel aus, aber die berechneten Ebenen sind nicht mehr stabil oder nicht mehr fachlich erklärbar.

In den folgenden Abschnitten spielt der Begriff SCC eine zentrale Rolle. Eine SCC (*Strongly Connected Component*, stark zusammenhängende Komponente) ist eine Menge von Knoten, in der jeder Knoten jeden anderen über gerichtete Abhängigkeitspfade erreichen kann. In einem solchen Zyklus können nicht alle Abhängigkeiten gleichzeitig nach unten laufen.

Ein korrektes Schichtlayout muss deshalb gleichzeitig mehrere Aussagen erfüllen:

1. Wenn $A \rightarrow B$ gilt, muss A oberhalb von B stehen.
2. Pakete müssen als Container sichtbar bleiben. Ein Paket darf nicht zufällig verschoben werden, nur weil eine einzelne enthaltene Klasse ein hohes Klassenlevel hat.
3. Klassen oder Pakete, die eine SCC bilden, müssen explizit behandelt werden, weil in einem Zyklus nicht alle Kanten gleichzeitig nach unten laufen können.

Diese Regeln sind einzeln gut lösbar. Zusammen erzeugen sie aber ein zirkuläres Abhängigkeitssystem zwischen den Ebenen der Berechnung selbst: Klassenlevel beeinflussen sichtbare Positionen, Paketcontainer strukturieren diese Positionen, und SCCs erzwingen Korrekturen an einer Ordnung, die man ohne Zyklusbehandlung schon berechnet hätte. Genau daraus entstehen die typischen Fehlermodi.

3.1 Retroaktive Korrektur

Ein naiver Algorithmus läuft über die Klassen und vergibt direkt Level. Er sieht zuerst Klasse A, setzt ihr Level und hebt damit auch das Paket von A an. Später findet er Klasse B und erkennt: A und B gehören zu derselben SCC und müssen gemeinsam behandelt werden. Jetzt müsste A korrigiert werden, und damit auch ihr Paket und möglicherweise weitere Pakete, die davon abhängen.

Das Problem ist nicht die Korrektur selbst. Das Problem ist, dass Teile des Layouts zu diesem Zeitpunkt bereits als fertig behandelt wurden. Ein Algorithmus, der Zyklenerkennung und Levelvergabe vermischt, muss rückwirkend an Stellen ändern, die er eigentlich schon verlassen hat.

3.2 Reihenfolgeabhängige Ergebnisse

Klassen kommen aus Jars. Die Reihenfolge, in der ein Tool Klassen sieht, folgt nicht der Architektur, sondern der Packreihenfolge des Jars, der Modulreihenfolge, dem Build-Tool oder Details des Dateisystems. Wenn ein Algorithmus Zyklen während dieser Traversierung nebenbei behandelt, hängt das Ergebnis von dieser zufälligen Reihenfolge ab.

Das ist gefährlich, weil der Fehler nicht sofort sichtbar sein muss. Die Grafik kann weiterhin ungefähr richtig aussehen. Trotzdem kann dieselbe Codebasis je nach Build-Konfiguration andere Level bekommen.

3.3 Vermischte Klassen- und Paketlevel

Ein naheliegender Fehler ist, das Level eines Pakets aus dem höchsten Level seiner enthaltenen Klassen abzuleiten. Das klingt plausibel, vermischt aber zwei verschiedene Fragen: Wo liegt eine Klasse im Klassengraphen, und wo liegt ein Paket in der Architekturhypothese?

Ein Paket ist nicht deshalb architektonisch hoch, weil es zufällig eine hoch eingeordnete Klasse enthält. Die zentrale Regel lautet:

Paketlevel entstehen aus Paketabhängigkeiten, nicht aus der maximalen Höhe einzelner Klassen.

Das klingt trivial, ist es aber nicht. Gerade weil Pakete Container für Klassen sind, liegt es nahe, ihre Position aus den enthaltenen Klassen abzuleiten. Genau dadurch vermischt man jedoch zwei Ebenen, die getrennt bleiben müssen. Erst wenn Paketlevel aus Paketabhängigkeiten entstehen, bleiben Pakete stabile architektonische Einheiten. Sonst genügt eine einzige Ausreißer-Klasse, um das Paket auf die falsche Ebene zu heben – und damit den Container unbrauchbar zu machen, der eigentlich Orientierung bieten soll.

3.4 Der große See

Die formal saubere Antwort auf Zyklen lautet: Alle Elemente einer SCC bekommen dasselbe Level. Für kleine Zyklen ist das akzeptabel. In realen Systemen können SCCs aber sehr groß werden. Dann landen Hunderte oder Tausende von Klassen auf derselben Ebene. Die Schichtung verschwindet; übrig bleibt ein flacher Block.

Damit wird der Zyklus zwar korrekt erkannt, aber nicht mehr brauchbar visualisiert. Ein Werkzeug muss große SCCs daher aufbrechen können. Entscheidend ist, dass die geschnittenen Kanten nicht verschwinden. Sie müssen sichtbar bleiben, weil genau sie erklären, warum die berechnete Ordnung nur als Architekturhypothese gilt.

4 Der Paketgraph als Architekturhypothese

Die Lösung besteht darin, Analyse, Ordnungsbildung und Darstellung konsequent zu trennen. Der Klassengraph bleibt das Rohmodell der tatsächlichen Abhängigkeiten. Er wird nicht verändert, nur weil für eine Schichtendarstellung ein DAG benötigt wird. Stattdessen erzeugt S202 aus diesem Graphen zusätzliche Sichten: SCCs für die Zyklerkennung, einen gewichteten Paketgraphen für die Architekturhypothese und eine hierarchische UI-Struktur für die Darstellung.

Mit Architekturhypothese ist dabei keine behauptete Soll-Architektur gemeint. Gemeint ist eine aus den beobachteten Abhängigkeiten abgeleitete Ist-Architektur: die Ordnung, die dem vorhandenen Code und seinen Abhängigkeiten am engsten entspricht, ohne zu verschweigen, wo diese Ordnung durch Zyklen oder Rückkanten gebrochen wird.

Die Paketordnung bildet dabei das zentrale Bindeglied. Sie wird nicht aus den höchsten Klassenlevels abgeleitet, sondern aus den aggregierten Paketabhängigkeiten. Diese Trennung ist der

entscheidende Schritt: Klassen werden innerhalb ihrer Pakete geordnet, Pakete dagegen nach ihren eigenen Paketbeziehungen. Dadurch kann ein Paket als Container stabil bleiben, auch wenn einzelne Klassen darin hoch oder niedrig liegen. Gleichzeitig liefert die Paketordnung ein Kriterium, um Back-Edges nicht beliebig auszuwählen, sondern als konkrete Verletzungen der erwarteten Ordnung zu kennzeichnen.

Damit verschwinden die oben beschriebenen Fehlermodi nicht durch einen einzelnen Algorithmusstrick, sondern durch eine klare Zuständigkeit der Berechnungsschritte: Erst werden Abhängigkeiten analysiert, dann wird eine plausible Ordnung bestimmt, danach wird die sichtbare Struktur aufgebaut, und zuletzt wird geprüft, ob die Darstellung ihre eigene Aussage einhält.

5 Von Klassenabhängigkeiten zur Paketordnung

Der flache Klassengraph ist in S202 nicht das sichtbare Layoutmodell. Er ist ein Analyseartefakt: Er dient dazu, konkrete Abhängigkeiten zu erfassen, SCCs zu erkennen, globale Klassenlevel zu berechnen und Klassenabhängigkeiten zu Paketbeziehungen zu aggregieren. Die sichtbare S202-Grafik entsteht danach aus der Paketstruktur und der lokalen Ordnung innerhalb der Paketcontainer.

Die Paketordnung entsteht aus den aggregierten Abhängigkeiten zwischen Paketen. Dafür werden die Klassenabhängigkeiten zunächst zu Paketabhängigkeiten aggregiert: Wenn Klassen aus Paket P Klassen aus Paket Q benutzen, entsteht eine gewichtete Kante $P \rightarrow Q$. Das Gewicht ist die Anzahl der aggregierten Nutzungen; für strukturelle Abhängigkeiten ohne Aufrufzählung wird ein Gewicht von 1 verwendet. Die Gegenrichtung wird ebenfalls gezählt.

Aus diesen Gewichten berechnet S202 für Pakete ein Rangmaß:

$$rank(P) = \frac{out(P) - in(P)}{\max(1, out(P) + in(P))}$$

$out(P)$ ist die Summe der gewichteten Kanten von P zu anderen betrachteten Paketen, $in(P)$ die Summe der gewichteten Kanten aus diesen Paketen nach P . Ein positiver Rang bedeutet: Das Paket benutzt mehr, als es selbst benutzt wird; es gehört tendenziell nach oben. Ein negativer Rang bedeutet: Das Paket wird eher benutzt und gehört tendenziell nach unten.

Eine Ordnungsentscheidung entsteht erst, wenn der Rangunterschied größer als eine Schwelle ε ist. In S202 wird dafür aktuell $\varepsilon = 0,1$ verwendet. Liegen zwei Pakete zu nah beieinander, bleiben sie konzeptionell Peers und damit auf demselben Level. Ein gegenseitiges Aufrufverhältnis von 100 : 101 ist deshalb keine belastbare Architekturentscheidung.

6 Vom Klassenzyklus zur Paketordnung

Abbildung 3 zeigt den zyklischen Teil des Example-Jars als klassischen Klassengraphen. Die beiden Teilgraphen für Notification und Payment enthalten jeweils Rückkopplungen.

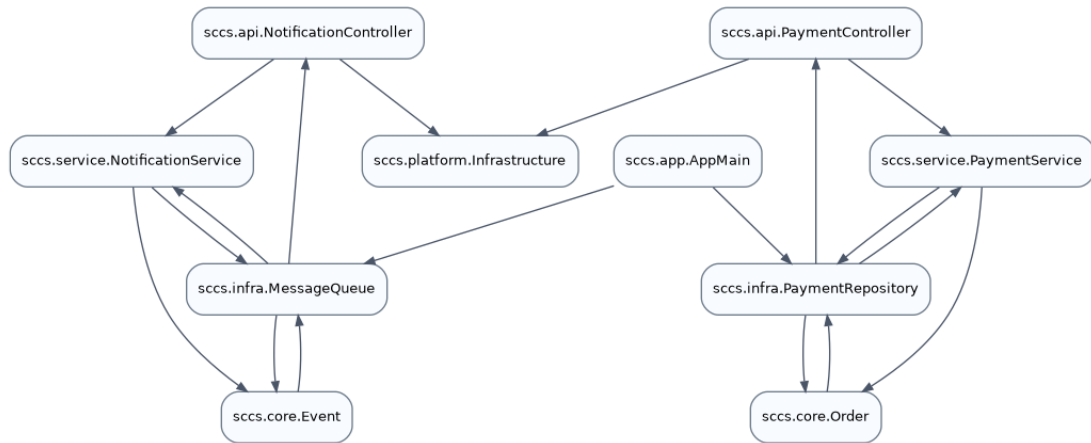


Abbildung 3: Zyklischer Ausschnitt aus `test-example-1.0.0.jar`. In jedem Zyklus muss mindestens eine Kante für die Levelberechnung geschnitten werden.

Die markierten zyklischen Bereiche sind zwei nicht-triviale SCCs: eine Notification-SCC mit `NotificationController`, `NotificationService`, `MessageQueue` und `Event`, sowie eine Payment-SCC mit `PaymentController`, `PaymentService`, `PaymentRepository` und `Order`. Das etablierte Verfahren zur Ermittlung solcher Komponenten ist der Tarjan-Algorithmus.

Damit stellt der Klassengraph im Zyklusfall die technische Frage: Welche Kante muss man ignorieren, damit aus der SCC wieder ein berechenbarer DAG für die Levelbildung wird? Der Klassengraph allein liefert dafür aber kein fachliches Kriterium. Erst die Paketordnung liefert dieses Kriterium: Eine als Back-Edge klassifizierte Kante ist dann keine beliebige Schnittkante im flachen Klassengraphen, sondern eine konkrete Abhängigkeit, die gegen die berechnete Paketordnung läuft und damit den Bruch in der Architekturhypothese sichtbar macht.

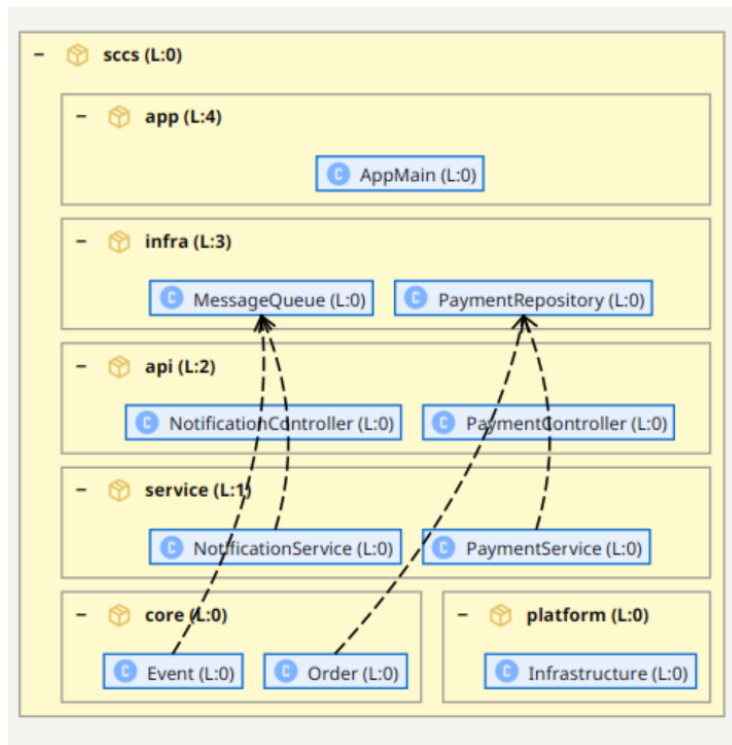


Abbildung 4: Zyklischer S202-Ausschnitt. Die gestrichelten Kanten sind Verletzungen: Sie laufen gegen die aus den Paketaabhängigkeiten abgeleitete Ordnung und bleiben als sichtbarer Befund der Architekturhypothese erhalten. Alle vier gestrichelten Kanten gehören zu den zwei SCCs und werden von S202 als Back-Edges für die Levelberechnung klassifiziert.

Die Begriffe Back-Edge, Verletzung und CUT beschreiben unterschiedliche Rollen:

Back-Edge Eine ausgewählte Kante innerhalb einer SCC, die für die Levelberechnung gesondert behandelt wird, damit der Berechnungsgraph azyklisch wird. Sie gehört zur Zyklusbehandlung.

Verletzung Eine konkrete Abhängigkeit, die nach der berechneten Ordnung nach oben läuft. Sie kann aus einer SCC stammen, muss es aber nicht. Im Beispiel sind alle sichtbaren Verletzungen zugleich Back-Edges; begrifflich bleibt eine Verletzung aber die sichtbare Ordnungsabweichung, während eine Back-Edge die Rolle der Kante in der Zyklusbehandlung beschreibt.

CUT Eine vom Benutzer markierte geplante Trennung eines konkreten Methodenaufrufs. Ein CUT ist keine automatische Layoutentscheidung, sondern ein Refactoring-Vorschlag, dessen Wirkung auf die SCC geprüft wird.

S202 versteckt den Zyklus daher nicht. Als Back-Edges klassifizierte Kanten bleiben als Verletzungen sichtbar, weil sie erklären, warum die Paketordnung nicht vollständig zu den konkreten Klassenabhängigkeiten passt. Dadurch wird die Abweichung nachvollziehbar statt beliebig.

7 Redundante Prüfung der Konsistenz

Die bisher beschriebenen Schritte zeigen bereits, warum die Umsetzung fehleranfällig ist. Es reicht nicht, einzelne Algorithmen isoliert mit Unit Tests abzusichern. Viele Fehler entstehen erst

durch das Zusammenspiel der Daten: konkrete Klassenabhängigkeiten, Paketstruktur, SCCs, aggregierte Paketgewichte und lokale Darstellungspositionen beeinflussen sich gegenseitig.

S202 prüft deshalb nicht nur einzelne Rechenschritte. Nachdem die komplette Pipeline gelaufen ist, wird das fertige Ergebnis redundant gegen Konsistenzregeln geprüft. Diese Regeln berechnen das Layout nicht neu und korrigieren auch nichts. Sie lesen nur das Ergebnis und fragen: Ist die erzeugte Darstellung in sich stimmig?

Die Prüfung betrachtet drei Ebenen:

1. **Klassengraph:** Für normale Klassenabhängigkeiten muss gelten: Wenn $A \rightarrow B$, dann steht A auf einem höheren Level als B . Ausnahmen müssen explizit klassifiziert sein, zum Beispiel als Back-Edge oder als Kante innerhalb einer SCC.
2. **Paketgraph:** Die berechnete Paketordnung muss zu den gewichteten Paketabhängigkeiten passen. Pakete mit höherem Rang müssen oberhalb der Pakete liegen, von denen sie stärker abhängen. Kanten, die beim Aufbrechen einer Paket-SCC geschnitten werden, dürfen in dieser Prüfung nicht wie normale dominante Kanten behandelt werden; sie müssen gesondert als Paket-Back-Edges klassifiziert sein.
3. **Sichtbare Darstellung:** Die sichtbare Anordnung muss zum berechneten Modell passen. Paketcontainer, lokale Klassenpositionen und verschachtelte Levels dürfen die berechnete Architekturhypothese nicht stillschweigend umkehren.

Diese Konsistenzregeln sind keine zusätzliche Optimierung und kein zweiter Layoutalgorithmus. Sie sind ein unabhängiger Plausibilitätscheck nach Abschluss der Berechnung. Genau dadurch finden sie Fehler, die in normalen Unit Tests leicht fehlen: ein Paketlevel wurde korrekt berechnet, aber lokal falsch dargestellt; eine Back-Edge wurde geschnitten, aber nicht als solche markiert; eine normale Kante läuft nach oben, ohne dass sie Teil einer SCC ist.

Der entscheidende Punkt ist Redundanz. Die Pipeline erzeugt die Darstellung. Die Regeln prüfen anschließend, ob diese Darstellung ihre eigene Aussage einhält. Damit wird aus einer hübschen Grafik ein überprüfbares Modell.

8 Von der Visualisierung zur Aktion

Eine berechnete Architekturhypothese ist besonders dann nützlich, wenn sie nicht nur einen Zustand beschreibt, sondern Veränderung planbar macht. S202 enthält dafür eine What-If-Analyse: Klassen und Pakete können per Drag and Drop in der Darstellung verschoben werden. Das Modell wird dadurch nicht sofort refaktoriert; stattdessen berechnet das Werkzeug, welche Abhängigkeiten in der neuen Anordnung nach oben laufen würden.

Die Verletzungen aktualisieren sich nach jeder Verschiebung. Dadurch wird sichtbar, ob eine geplante Umordnung die Architektur konsistenter macht oder neue Probleme erzeugt. Das ist vor allem in Situationen wichtig, in denen das Werkzeug keine eindeutige Ordnung mehr findet: große SCCs, symmetrische Paketabhängigkeiten oder historisch gewachsene Kopplungen lassen sich nicht immer automatisch sinnvoll auflösen.

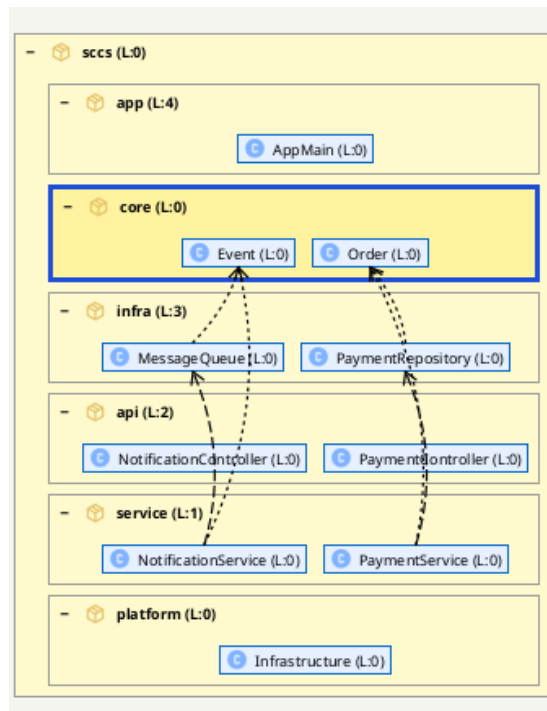


Abbildung 5: What-If-Verschiebung: Das Paket `core` wird aus Level 0 zwischen Level 3 und Level 4 verschoben. Dadurch entstehen sechs Verletzungen, weil bestehende Abhängigkeiten nun gegen die gewählte Anordnung laufen.

Abbildung 5 zeigt genau diese Arbeitsweise: Die manuelle Anordnung entspricht nicht mehr der aus den Abhängigkeiten berechneten Ordnung. Sie kann trotzdem eine gewünschte Zielarchitektur ausdrücken, etwa wenn `core` fachlich oberhalb der Infrastruktur liegen soll. S202 behandelt diese Entscheidung deshalb nicht als Fehler der Darstellung, sondern macht ihre Kosten sichtbar: In diesem Fall müssten sechs konkrete Beziehungen refaktoriert oder entkoppelt werden, damit die Abhängigkeiten wieder zur gewählten Schichtung passen.

Ergänzend dazu gibt es eine manuelle CUT-Logik. Back-Edges können gezielt als geplante Schnitte markiert werden, und das Werkzeug kann anschließend prüfen, ob diese Schnitte die betroffene SCC tatsächlich auflösen. Der Benutzer sieht nicht nur, welche Kante als Bruch der Ordnung plausibel wäre, sondern kann die Wirkung dieses Schnitts auf den Zyklus überprüfen.

Diese CUT-Logik arbeitet feiner als die sichtbare Klassen- oder Paketkante. Geschnitten wird auf Methodenaufabeebene: Nicht zwingend die gesamte Beziehung zwischen zwei Klassen verschwindet, sondern der konkrete Methodenaufwurf, der die zyklische Abhängigkeit verursacht. Dadurch passt das Feature besser zur praktischen Refactoring-Arbeit, weil ein Zyklus oft durch wenige konkrete Aufrufe entsteht, nicht durch die komplette fachliche Beziehung zweier Klassen.

9 Umsetzung auf konzeptioneller Ebene

Die Umsetzung trennt die Berechnung bewusst in mehrere Phasen. Entscheidend ist, dass keine Phase eine sichtbare Position festlegt, bevor die Informationen vorliegen, die diese Position begründen.

1. **Klassengraph aufbauen:** Aus dem Bytecode wird ein gerichteter Klassengraph erzeugt. Dieser Graph bleibt das fachliche Rohmodell der Analyse.

2. **SCCs erkennen:** Auf dem Klassengraphen werden stark zusammenhängende Komponenten mittels Tarjan ermittelt. Das Ergebnis ist noch kein Layout, sondern die Information, welche Klassen azyklisch einordenbar sind und welche zu Zyklen gehören.
3. **Paketgraph ableiten:** Aus den Klassenabhängigkeiten werden Paketabhängigkeiten aggregiert und gewichtet.
4. **Paketordnung berechnen:** Aus den gewichteten Ein- und Ausgängen wird das Rangmaß $rank(P)$ berechnet. Daraus entsteht die vertikale Paketordnung. Kleine Rangunterschiede werden als Peer-Beziehung behandelt.
5. **Back-Edges bestimmen:** Innerhalb jeder SCC werden die Kanten ausgewählt, die für die Zyklusentfernung ausreichen. Diese Kanten sind Back-Edges für die Levelberechnung und bleiben zugleich als Verletzungen sichtbar, wenn sie nach der berechneten Ordnung nach oben laufen. *Geschnitten* bedeutet dabei nicht, dass die Kante aus dem Modell verschwindet. Sie wird nur für den DAG der Levelberechnung ausgenommen und bleibt als Befund erhalten.
6. **Lokale Schichtung aufbauen:** Erst danach wird die sichtbare Baumstruktur erzeugt. Innerhalb jedes Containers werden nur die direkten Kinder angeordnet. Dadurch wird die globale Paketordnung nicht mit lokalen Klassenlevels vermischt.
7. **Konsistenz prüfen:** Zum Schluss wird die fertige Darstellung redundant gegen die Konsistenzregeln geprüft. Normale Abhängigkeiten, SCCs, Back-Edges und Verletzungen werden unterschieden und klassifiziert.

Kurz gefasst:

```
Bytecode -> Klassengraph -> SCCs -> Paketgraph -> Paketordnung
          -> Back-Edges -> sichtbare Schichtung -> Konsistenzregeln
```

10 Fazit

S202 berechnet aus den bestehenden Abhängigkeiten die plausibelste Hypothese der Ist-Architektur. Das Ergebnis ist eine überprüfbare hierarchische Schichtendarstellung: Klassenabhängigkeiten liefern den Graphen, Paketabhängigkeiten liefern eine belastbare Ordnung, Back-Edges machen notwendige Brüche sichtbar, und Konsistenzregeln prüfen, ob die fertige Darstellung ihre eigene Aussage einhält.

Der wichtigste Beitrag liegt deshalb nicht in einer schöneren Grafik, sondern in der Verbindung von Darstellung, Erklärung und Veränderbarkeit. Die Schichtung zeigt eine mögliche Ordnung. Die markierten Befunde erklären, wo diese Ordnung durch konkrete Abhängigkeiten gebrochen wird. What-If-Verschiebungen und manuelle CUTs machen sichtbar, welche Refactorings diese Brüche reduzieren oder Zyklen tatsächlich auflösen können.

Damit ist die Visualisierung mehr als ein Layout. Sie ist ein erklärbares Arbeitsmodell: Jede Position und jede hervorgehobene Kante muss auf eine nachvollziehbare Entscheidung in der Berechnung zurückführbar sein, und jede geplante Änderung kann gegen dieselbe Ordnung geprüft werden.